



DRUM: A Real Time Detector for Regime Shifts in Data Streams via an Unsupervised, Multivariate Framework

Adnan Bashir^(✉)  and Trilce Estrada 

Computer Science Department, University of New Mexico, Albuquerque, NM, USA
{abashir,estrada}@cs.unm.edu

Abstract. In this work we present DRUM, an unsupervised approach that is based on statistical properties of multivariate data streams to identify regime shifts in real time. DRUM processes streams in small chunks, learns their statistical properties, and makes generalizations as time goes by. We show how this straightforward approach requires minimal computation and reaches state of the art accuracy, making it ideal for embedded and cyber physical systems.

Keywords: Multivariate data streams · Change point detection · Statistical analysis · Unsupervised · Real time · Online detection

1 Introduction

A regime shift refers to a sudden or significant change over time in the behavior of a system. It can occur in various natural and social systems, ranging from ecosystems and climate patterns to economic and social systems. Regime shifts in cyber physical systems can indicate potential problems or pattern changes that need to be dealt with in real time [3]. Regime Shift Detection (RSD), or Change Point Detection (CPD) refers to the identification of such changes in the underlying distribution of data streams [5]. Detection of regime shifts is of vital importance in many real-world problems such as weather monitoring, early detection of cyber security threats, medical monitoring, speech analysis, market analysis, human activity monitoring and many more [23]. For dynamic cyber physical systems, real time detection of regime shifts is essential for prevention and mitigation of system failures. Understanding when a regime shift is underway enables informed decision making, improved risk management, and accurate policy development, leading to increased resilience of the overall system.

In this paper we present DRUM (A real time **D**etector for **R**egime shifts in data streams via an **U**nsupervised and **M**ultivariate framework). DRUM has the following properties that make it suitable for solving regime shift detection in data streams: **Unsupervised** - Our method doesn't require labels or human intervention. **Real time responsiveness** - Due to very low computation, it

offers minimum lag in detecting change points. **Online** - Our method processes data as a stream and does not require retraining or batch processing to adapt to changes over time. **General** - Our method has been tested on various univariate and multivariate datasets, and it was able to perform as well or with higher accuracy as other frameworks.

2 Related Work

Offline Methods - Offline CPD refers to those methods that require a complete data stream for detection [5]. Numerous offline CPD algorithms have been proposed in the past few decades. CUSUM method [21] was one of the earliest change point detectors that identified points in time series with their cumulative sum exceeding a threshold value. DRE-CUSUM [2] is built upon CUSUM which splits the data stream from an arbitrary point and compares the distribution on both sides to detect any change. Binary segmentation was introduced as an approximate change point detection method that greedily splits the data stream into disjointed segments based on a cost function. To reduce the higher time complexity of binary segmentation, pruning was added [14]. Wild Binary Segmentation extended the original binary segmentation method by applying CUSUM on subsets of data stream [10].

Online Methods - Online CPD refers to those methods that process the data as it arrives. They can be viewed as ϵ -real time algorithms [5]. Where ϵ is the delay in timestamps required by the algorithm to accurately detect a change point. To incorporate multivariate data streams and online requirement of CPD, Binary Online Changepoint Detection (BOCD) was proposed [1]. BOCD introduced a term called run-length, which denotes data stream since the most recent change point. BOCD approximates the probability distribution over run-length and compares it with previously approximated distribution for change point detection. Lately, several methods were proposed that were based on BOCD namely BOCPDMS [16] and RBOCPDMS [15]. We have compared all variants of BOCP due to its online detection ability.

Supervised Methods - Supervised learning algorithms have proven to be very effective when it comes to change point detection but they require labeled data and a training phase. Traditional machine learning methods like SVM [8], random forests [6], decision trees [22], hidden Markov models [20], nearest neighbors [19], and other binary classifiers [4] have been used to deal with this problem as a supervised classification problem. Among them, RFPOP [9] detects change points in an offline setting by engineering the cost function via dynamic programming algorithm. Another such approach is PELT [14] which minimizes the detection delay in univariate time series. Facebook has proposed PROPHET [25] which works as a regression model which requires domain expertise for tuning. This is not feasible for unsupervised and online applications.

Deep Learning and Neural Network-based Models - Neural Networks have been readily used for time series analysis. But they also suffer some limitations when it comes to online CPD. As stated by Zhang et al. [27], the network

structure, training method and sample data may affect the performance and accuracy of Neural Network based methods. Also training Neural Networks is a time-consuming process and usually requires a big chunk of data in training. For example, GAN (Generative Adversarial Networks [11]) based models require entire variable set concurrently to tune the hidden layer parameters (such as MAD-GAN [18]). Re-tuning these models is again a time-consuming process that compromises the real time response of these detectors. Hence a majority of Neural Network based CPD algorithms outperform when trained in an offline setting.

3 DRUM

In this section we describe our approach (DRUM) which consists of two modules: a scoring module, which computes statistics of the individual data streams; and a detection module, which quantifies the rate of change and determines if a regime shift has occurred.

Scoring Module: this module takes a multivariate data stream and calculates our Lobo Change Score (LCS), which is the weighted sum of change in mean (Δm), change in standard deviation (Δs), and change in fluctuations across the running mean (Δfrm) between two data windows of the data stream. Formally, the data stream is an infinite sequence of values generated by an underlying system and is denoted as $S = \{x_1, x_2, \dots, x_{t-1}, x_t, x_{t+1}, \dots\}$, where for a multivariate time-series with n variables, x_t is the n -dimensional data vector at time t [26]. LCS is computed for small chunks of data that we call windows. That is, given variable i of the stream, for a window of size $d + 1$ starting in position j , our algorithm processes the sequence $w_{i,[j,j+d]} = \{x_{i,j}, x_{i,j+1}, x_{i,j+2}, \dots, x_{i,j+d}\}$. LCS is composed of three components:

$$\Delta m_{i,j,k} = |m(w_{i,[j,j+d]}) - m(w_{i,[k,k+d]})| \quad (1)$$

$$\Delta s_{i,j,k} = |s(w_{i,[j,j+d]}) - s(w_{i,[k,k+d]})| \quad (2)$$

$$\Delta frm_{i,j,k} = |frm(w_{i,[j,j+d]}) - frm(w_{i,[k,k+d]})| \quad (3)$$

where for two windows: $w_{i,[j,j+d]}$ starting at position j , and $w_{i,[k,k+d]}$ starting at position k ; both of them corresponding to variable i , $\Delta m_{i,j,k}$ is the absolute difference between their mean; $\Delta s_{i,j,k}$ is the absolute difference in their standard deviation; and Δfrm is the absolute difference of fluctuation across the running mean. frm is computed by counting the number of intersections between the running mean and the signal itself within the current window. Then, LCS for windows $[j, j + d]$ and $[k, k + d]$ is the weighted sum of the three components described above, as shown by Eq. 4:

$$LCS_{j,k} = \alpha \sum_{i=1}^n \Delta m_{i,j,k} + \beta \sum_{i=1}^n \Delta s_{i,j,k} + \gamma \sum_{i=1}^n \Delta frm_{i,j,k} \quad (4)$$

where α , β and γ are weight coefficients, used to differentiate the contribution of the three components. They can have values between 0 and 1, these values can be updated adaptively depending on the data characteristics, or can be kept constant to simplify hyperparameter selection. As shown in Algorithm 1, we use two types of windows for calculating LCS. LCS_D is calculated using disjointed windows, that is, windows of size $d + 1$ that do not overlap, they track coarse changes over a period of time. While LCS_S uses consecutive sliding windows to detect with finer precision when a change occurs. Figure 1 visually explains the two types of windows.

Algorithm 1: DRUM

Input: Multivariate data stream S ; window length $d + 1$; α, β, γ
Output: LCS_D, LCS_S
 $t \leftarrow 0; l \leftarrow t + 1; j \leftarrow d + 1; k \leftarrow 1;$
 $LCS_D = \{\}, LCS_S = \{\}$
while $S \neq \{\}$ **do**
 while $t + 1 \leq d$ **do**
 $\forall_i w_{i,[k,k+d]} \leftarrow x_{i,t}$ /* gather data for window $[k, k + d]$ */
 end
 $t++$ /* use t as the timestamp counter */
 /* compute LCS_D for disjoint windows j and k */
 if $j \leq t \leq j + d$ **then**
 $\forall_i w_{i,[j,j+d]} \leftarrow x_{i,t}$ /* gather data for window $[j, j + d]$ */
 end
 if $t = j + d$ **then**
 $LCS_D \leftarrow \forall_i \alpha \Delta m_{i,j,k} + \beta \Delta s_{i,j,k} + \gamma \Delta frm_{i,j,k}$
 $k \leftarrow j$
 $j \leftarrow j + d + 1$
 end
 /* compute LCS_S for consecutive sliding windows ending at t */
 update $\forall_i w_{i,[t-d,t-1]}$
 update $\forall_i w_{i,[t-d+1,t]}$
 $LCS_S \leftarrow \forall_i \alpha \Delta m_{i,t-d,t-d+1} + \beta \Delta s_{i,t-d,t-d+1} + \gamma \Delta frm_{i,t-d,t-d+1}$
 $LCS_S.append(LCS_S)$
 if $detectChange(LCS_D)$ **then**
 return $maxChange(LCS_S)$
 end
end

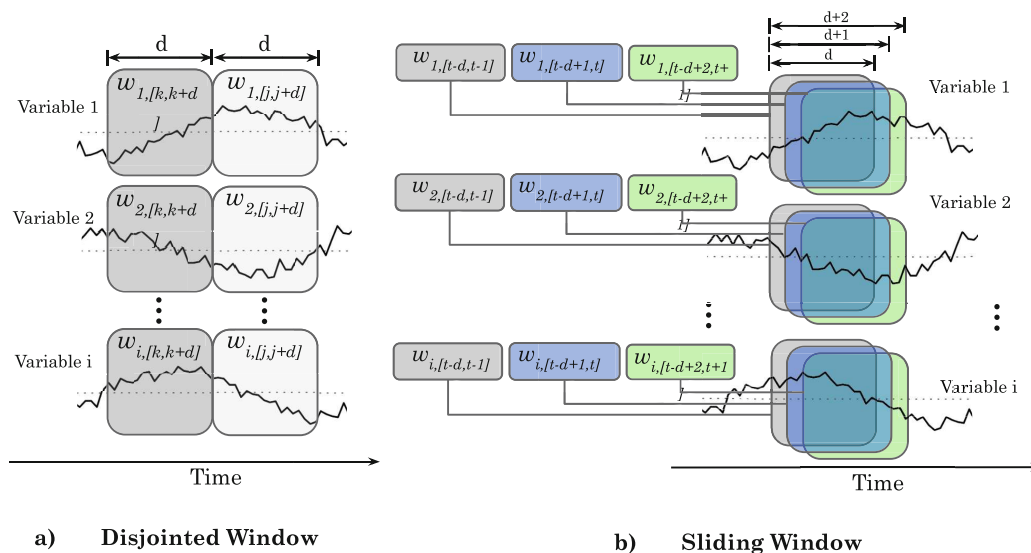


Fig. 1. Visualization of a) Disjointed Windows and b) Sliding Windows

Detection Module: Once LCS is calculated for disjointed (LCS_D) and sliding windows (LCS_S), it is evaluated by the detection module. The first step in detecting regime shift is to identify the window in which the change has occurred. If LCS_D has increased or decreased by 5% or more, the window is marked as a change point window. Once the change window is identified, we use LCS_S to further identify the specific timestamp with the maximum change score. This timestamp is returned as a change point.

4 Evaluation

To evaluate the accuracy and performance of our method, we compared its behavior on datasets labeled for CPD (see Table 1) with respect to other state of the art methods (see Table 2). Additional details regarding these methods were presented in Sect. 2. We used three metrics for comparison: NAB score, F1 score, and execution time. Below we present the results of such comparisons.

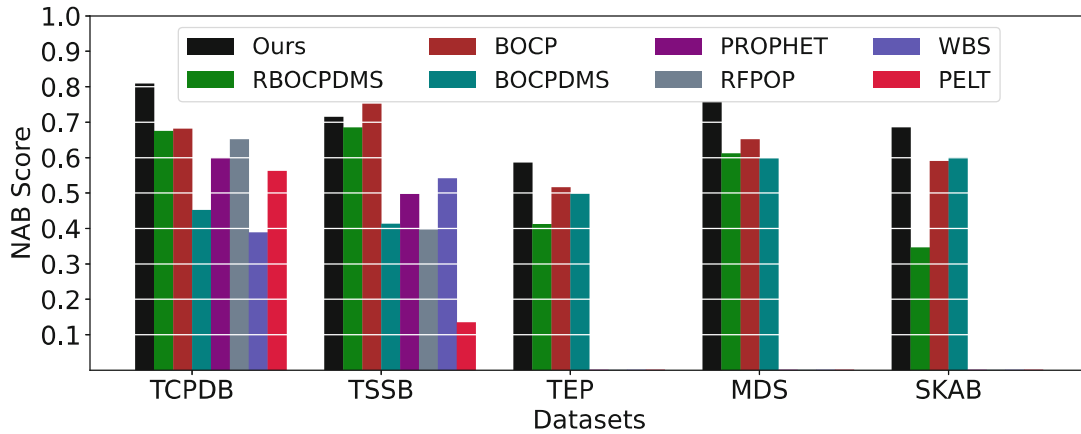
Table 1. Datasets and Benchmarks used for Evaluation

DATASET	Description	# of Variables	Length	# of Series
TCPD [7]	Real-world dataset for bench-marking CPD algorithms	1–5	15–991	42
TSSB [24]	Subset of UCR TS datasets	1	572–15970	66
SKAB [13]	Sensors on a testbed	8	666–1327	34
TEP [12]	Industrial chemical process	52	500–960	22
MDS	Mesa Del Sol microgrid	18	518960	1

Table 2. Properties of CPD frameworks used for evaluation

METHOD	Year	Online	No Training	Unsupervised	Multivariate
DRUM (ours)	2023	✓	✓	✓	✓
RFPOP [9]	2019	✗	✓	✗	✗
RBOCPDMS [16]	2018	✓	✓	✓	✓
PROPHET [25]	2018	✗	✗	✗	✗
BOCPDMS [15]	2018	✓	✓	✓	✓
WBS [10]	2012	✗	✓	✓	✗
PELT [14]	2012	✗	✓	✓	✗
BOCPD [1]	2007	✓	✓	✓	✓

NAB Score. NAB [17] was proposed in 2015 by Numenta¹. It was the first anomaly and change point detection benchmark designed to evaluate unsupervised and real time algorithms. It rewards early detection and uses a scaled sigmoidal function to score false positives and false negatives. Figure 2 shows the NAB standard profile score. Our approach outperforms other methods in four out of five datasets (TCPBD, TEP, MDS, SKAB), and performs competitively in TSSB. It is important to note that NAB score allows the framework to perform a change point detection within a window (by default it is 5% of the length of time series).

**Fig. 2.** NAB Standard Profile Score, Empty bars show that the framework is not multivariate capable

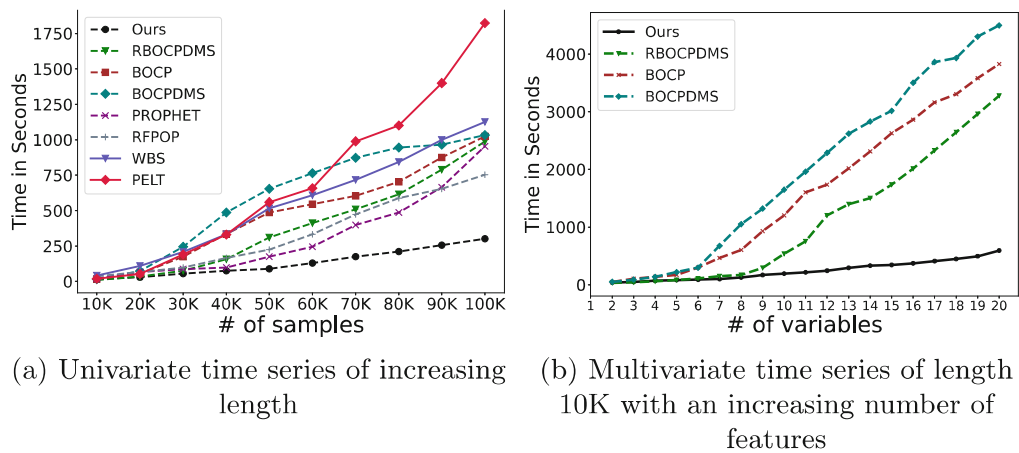
F1 Score. When a framework has a binary output (in this case change or no change), F1-Score has been readily used by the research community to evaluate such frameworks. F1 scores for the different methods are presented in Table 3 with the highest score per dataset in bold. Our approach outperforms other methods in four out of five datasets.

¹ <https://numenta.com/>.

Table 3. F1 Score. Empty fields mean that the framework is not designed for multi-variate time series data

METHOD	TCPDB	TSSB	SKAB	TEP	MDS
DRUM (ours)	0.804	0.715	0.634	0.664	0.714
BOCPD [1]	0.818	0.345	0.568	0.608	0.640
BOCPDMS [15]	0.620	0.589	0.618	0.652	0.687
RBOCPDMS [16]	0.447	0.548	0.623	0.325	0.686
PELT [14]	0.787	0.563	-	-	-
PROPHET [25]	0.534	0.446	-	-	-
WBS [10]	0.533	0.214	-	-	-
RFPOP [9]	0.531	0.197	-	-	-

Execution Time. We also compared the runtime of CPD frameworks both on univariate and multivariate time series. All Experiments were run on an Intel Core i7-8750H @ 2.20 GHz with 6 cores and 32 GB of RAM. Implementation was done using Python 3.7 on Windows 11. Fig. 3a shows the execution time for univariate time series from 10,000 to 100,000 instances, sampled at intervals of 10,000. Our approach has the lowest execution time with approximately linear behavior, while others show an exponential response. Figure 3b shows the execution time for multivariate time series with a fixed length of 10,000 but with the number of features ranging from 2 to 20. Computation time for BOCPD and its variants depends on change point location: if a change point is not detected more samples are stored to approximate the distribution of the data stream. This results in longer execution over time. While our approach always used a constant amount of data and exhibits linear scalability.

**Fig. 3.** Comparison of execution time

5 Conclusion

In this work, we present an unsupervised change point detector that works on streams and scales linearly with the number of variables being tracked. It requires minimal computations and can be applied toward detecting different events happening in a system in real time. Such events include system failures, concept drift, and state transitions. Since it is lightweight, it can be run on IoTs with low compute power.

Acknowledgments. This work was supported by National Science Foundation (NSF) EPSCoR grant number OIA-1757207.

References

1. Adams, R.P., MacKay, D.J.C.: Bayesian Online Changepoint Detection (2007). <http://arxiv.org/abs/0710.3742>
2. Adiga, S., Tandon, R.: Unsupervised change detection using dre-cusum. arXiv preprint [arXiv:2201.11678](https://arxiv.org/abs/2201.11678) (2022)
3. Ahamed, R., Lavin, A., Purdy, S., Agha, Z.: Unsupervised real-time anomaly detection for streaming data. *Neurocomputing* **262**, 134–147 (2017). <https://doi.org/10.1016/j.neucom.2017.04.070>
4. Aminikhanghahi, S., Cook, D.J.: A survey of methods for time series change point detection. *Knowl. Inf. Syst.* **51**(2), 339–367 (2016). <https://doi.org/10.1007/s10115-016-0987-z>
5. Aminikhanghahi, S., Wang, T., Cook, D.J.: Real-time change point detection with application to smart home time series data. *IEEE Trans. Knowl. Data Eng.* **31**(5), 1010–1023 (2019). <https://doi.org/10.1109/TKDE.2018.2850347>
6. Athey, S., Tibshirani, J., Wager, S.: Generalized random forests (2019)
7. van den Burg, G.J., Williams, C.K.: An evaluation of change point detection algorithms. arXiv, pp. 1–33 (2020)
8. Camci, F.: Change point detection in time series data using support vectors. *Int. J. Pattern Recognit. Artif. Intell.* **24**(01), 73–95 (2010)
9. Fearnhead, P., Rigaill, G.: Changepoint detection in the presence of outliers. *J. Am. Stat. Assoc.* **114**(525), 169–183 (2019)
10. Fryzlewicz, P.: Wild binary segmentation for multiple change-point detection. *Ann. Stat.* **42**(6), 2243–2281 (2014)
11. Goodfellow, I., et al.: Generative adversarial nets. In: *Advances in Neural Information Processing Systems*, pp. 2672–2680 (2014)
12. Katser, I., Kozitsin, V., Lobachev, V., Maksimov, I.: Unsupervised offline change-point detection ensembles. *Appl. Sci.* **11**(9), 1–19 (2021). <https://doi.org/10.3390/app11094280>
13. Katser, I.D., Kozitsin, V.O.: Skoltech anomaly benchmark (SKAB) (2020). <https://www.kaggle.com/dsv/1693952>. <https://doi.org/10.34740/KAGGLE/DSV/1693952>
14. Killick, R., Fearnhead, P., Eckley, I.A.: Optimal detection of changepoints with a linear computational cost. *J. Am. Stat. Assoc.* **107**(500), 1590–1598 (2012)
15. Knoblauch, J., Damoulas, T.: Spatio-temporal Bayesian on-line changepoint detection with model selection. In: *International Conference on Machine Learning*, pp. 2718–2727. PMLR (2018)

16. Knoblauch, J., Jewson, J.E., Damoulas, T.: Doubly robust Bayesian inference for non-stationary streaming data with divergences. In: *Advances in Neural Information Processing Systems*, vol. 31 (2018)
17. Lavin, A., Subutai, A.: Numenta anomaly benchmark. In: *International Conference on Machine Learning and Applications*, vol. 14 (2015)
18. Li, D., Chen, D., Shi, L., Jin, B., Goh, J., Ng, S.K.: MAD-GAN: multivariate anomaly detection for time series data with generative adversarial networks. *arXiv*, vol. 1, pp. 703–716 (2019)
19. Liu, Y.W., Chen, H.: A fast and efficient change-point detection framework based on approximate k -nearest neighbor graphs. *arXiv preprint [arXiv:2006.13450](https://arxiv.org/abs/2006.13450)* (2020)
20. Miller, D.J., Ghalyan, N.F., Mondal, S., Ray, A.: Hmm conditional-likelihood based change detection with strict delay tolerance. *Mech. Syst. Signal Process.* **147**, 107109 (2021)
21. Page, E.S.: Continuous inspection schemes. *Biometrika* **41**(1/2), 100–115 (1954)
22. Reddy, S., Mun, M., Burke, J., Estrin, D., Hansen, M., Srivastava, M.: Using mobile phones to determine transportation modes. *ACM Trans. Sen. Netw.* **6**(2) (2010). <https://doi.org/10.1145/1689239.1689243>
23. Reeves, J., Chen, J., Wang, X.L., Lund, R., Lu, Q.Q.: A review and comparison of changepoint detection techniques for climate data. *J. Appl. Meteorol. Climatol.* **46**(6), 900–915 (2007)
24. Schäfer, P., Ermschaus, A., Leser, U.: ClaSP - time series segmentation. In: *CIKM* (2021)
25. Taylor, S.J., Letham, B.: Business time series forecasting at scale. *PeerJ Preprints* 5:e3190v2 **35**(8), 48–90 (2017)
26. Tran, D.H.: Automated change detection and reactive clustering in multivariate streaming data. In: *2019 IEEE-RIVF International Conference on Computing and Communication Technologies (RIVF)*, pp. 1–6. IEEE (2019)
27. Zhang, G., Patuwo, B.E., Hu, M.Y.: Forecasting with artificial neural networks: the state of the art. *Int. J. Forecast.* **14**(1), 35–62 (1998). [https://doi.org/10.1016/S0169-2070\(97\)00044-7](https://doi.org/10.1016/S0169-2070(97)00044-7)